GENAI INFRASTRUCTURE & LLMOPS PREP

15+

Memory, Databases & Caching for GenAI

INTERVIEW QUESTIONS

WHAT YOU'LL MASTER INSIDE

Every Answer Explained Simply

Short & Long-Term Memory • Relational vs Vector DBs • Document Ingestion • HNSW & Hybrid Search • Multi-Tenant Isolation

1. MEMORY DESIGN

Q1. How would you design short-term and long-term memory for an AI agent performing multi-step tasks?

Short-term memory holds working context in the LLM context window; long-term memory stores durable facts across sessions in external databases.

Memory Type	Short-Term Memory	Long-Term Memory
Storage Layer	Active Context Window (RAM)	Vector + Relational DBs
Primary Content	Task plan, tool outputs, steps	User facts, past decisions, domain knowledge
Key Constraint	Strict token limit (e.g. 128k)	Retrieval latency & relevance
Management	Token counting & step compression	Semantic search & recency decay

THE MEMORY GRADUATION HANDOFF

When a task concludes, durable findings (e.g., user preferences, verified workflow solutions) **graduate** from working context into long-term vector/relational tables so future sessions retrieve them automatically.

Check caption for complete AI Engineer Interview Kit.

1. MEMORY DESIGN

Q2. What architectural challenges arise when storing conversation history for long-running chat sessions?

Long-running sessions create compounding engineering hurdles around storage throughput, token window saturation, and state tracking.

1. **Storage Throughput at Scale:** Millions of active turns require relational indexing (`user_id`, `session_id`, `turn_number`) optimized for high write concurrency and slice reads.

2. **Context Injection Strategy:** You cannot load 300 turns. Combine **Recency** (last 5-10 turns verbatim) + **Semantic Search** (top-k relevant turns) + **Anchor turns** (first 2 turns).

3. **Structured Session State:** Beyond chat text, store decisions and preferences in structured JSON records injected as dense summaries rather than raw dialogue logs.

BEST PRACTICE ARCHITECTURE

Never pass raw full history. Store immutable turns in SQL, index embeddings in a vector store, and assemble context dynamically per turn.

Check caption for complete AI Engineer Interview Kit.

1. MEMORY DESIGN

Q3. How would you prevent conversation memory from exceeding the model's context window?

Implement proactive token budgeting and a hierarchical compression strategy before hitting the prompt context limit.

```
def assemble_context(system, history, query, max_tokens=100000):
  budget = max_tokens - 2000 # reserve output buffer
  reserved = count_tokens(system) + count_tokens(query)
  return fit_history_to_budget(history, budget - reserved)
```

3-TIER COMPRESSION HIERARCHY

- **Tier 1: Tool Output Summarization:** Replace 500-token JSON tool returns with 20-token key findings once consumed.
- **Tier 2: Rolling Summarization:** Every 20 turns, run a background LLM job to collapse older blocks into a 1-paragraph summary.
- **Tier 3: Truncation Gate:** Retain only the rolling summary + last N turns + top-k semantically matched turns.

Check caption for complete AI Engineer Interview Kit.

1. MEMORY DESIGN

Q4. What strategies allow an AI system to remember important facts while discarding irrelevant context?

Distinguish high-value signal from conversational chatter using explicit extraction, decay functions, and async consolidation.

```
// Structured Fact Extraction Output
{
  "user_preferences": ["prefers bullet points", "fintech industry"],
  "established_facts": ["project launch: March 15"],
  "user_decisions": ["selected GPT-4o for production"]
}
```

1. **Explicit Importance Signals:** Extract statements with phrases like "I always prefer" or "remember that" directly into structured JSON tables.

2. **Decay-Based Forgetting:** Weight vector retrieval scores by `Semantic Similarity × Recency Decay` so unaccessed older notes fade.

3. **Async Consolidation:** Periodically deduplicate memory; mark old preferences as "superseded" when users change directions.

Check caption for complete AI Engineer Interview Kit.

1. MEMORY DESIGN

Q5. How would you design memory systems for multi-user AI applications where sessions must remain isolated?

Multi-tenant isolation requires mandatory hard filtering at the database access layer—never rely solely on LLM prompt instructions.

```
def get_user_memories(user_id: str, query_emb: list) -> list:
  return vector_store.search(
    embedding=query_emb,
    mandatory_filters={"user_id": user_id}, # infrastructure layer
    top_k=10
  )
```

DEFENSE-IN-DEPTH ISOLATION STACK

- **Tenant Scoped Schema:** Every database row & vector metadata payload must contain indexed `user_id` and `tenant_id` keys.
- **Redis Cache Namespacing:** Namespace keys strictly as `session:{user_id}:{session_id}:state` to prevent key collisions.
- **System Prompt Scoping:** System prompts instruct the LLM to assist only the current user ID as an added guardrail.

Check caption for complete AI Engineer Interview Kit.

2. DATABASES FOR GENAI

Q6. When designing a GenAI system, how do you decide between relational, document, and vector databases?

Modern GenAI systems use a polyglot persistence architecture combining structured SQL with semantic vector indexes.

DB Type	Optimal AI Workload	Key Technologies
Relational	ACID users, billing, audit logs, metadata filters	PostgreSQL (pgvector)
Document	Schema-flexible agent execution traces, logs	MongoDB, DynamoDB
Vector DB	Large-scale approximate nearest neighbor search	Pinecone, Qdrant, Milvus

PRODUCTION POLYGLOT STACK

In practice, **PostgreSQL + pgvector** handles up to 10M vectors alongside structured metadata. Dedicated vector databases scale when datasets exceed tens of millions of embeddings.

Check caption for complete AI Engineer Interview Kit.

2. DATABASES FOR GENAI

Q7. What advantages do vector databases provide compared to traditional databases in AI applications?

Purpose-built vector infrastructure delivers specialized approximate search algorithms and distributed embedding management.

OPTIMIZED ANN SEARCH

Hardware-tuned HNSW, DiskANN, and IVF indexes achieve sub-50ms latency across 100M+ vectors.

SINGLE-STAGE HYBRID

Execute combined semantic vector similarity + structured metadata pre-filtering in one pipeline.

LIFECYCLE MANAGEMENT

Built-in sharding, replication, dynamic quantization, and incremental background graph updates.

HORIZONTAL SCALE

Distributed graph partitioning designed specifically for high-dimensional floating-point vectors.

Check caption for complete AI Engineer Interview Kit.

2. DATABASES FOR GENAI

Q8. How would you structure a database schema to support document ingestion for RAG pipelines?

Separate parent document metadata from chunk embeddings to support atomic invalidation, deduplication, and ACL filters.

```
CREATE TABLE documents (id UUID PRIMARY KEY, file_hash VARCHAR(64),
status VARCHAR(20), version INT);
CREATE TABLE document_chunks (
  id UUID PRIMARY KEY, document_id UUID REFERENCES documents(id),
  chunk_index INT, embedding_model VARCHAR(100), embedding_version
INT,
  vector_store_id VARCHAR(200)
);
```

WHY THIS SCHEMA WORKS

- **Cascade Invalidation:** Linking chunks to `document_id` lets you supersede all chunks atomically on document updates.
- **Embedding Model Tracking:** Tracking `embedding_model` allows incremental migrations when upgrading embedding dimensions.
- **ACL Governance:** Join against a `document_permissions` table at retrieval time to enforce user read access.

Check caption for complete AI Engineer Interview Kit.

2. DATABASES FOR GENAI

Q9. What challenges arise when storing millions of embeddings in production systems?

At billion-scale, bottlenecks shift from computation to RAM capacity, index build latency, and disk IOPS trade-offs.

Index Strategy	Memory Footprint	Latency & Scale
In-Memory HNSW	High (Full graph in RAM)	Lowest latency (<10ms), costly at scale
DiskANN / SSD	Low (Graph stored on NVMe)	High capacity, slight IO overhead
IVF-PQ	Compressed (8x–32x smaller)	Scales to billions, minor recall drop

PRODUCT QUANTIZATION (PQ) ADVANTAGE

When full-precision float32 embeddings exceed RAM capacity, apply **Product Quantization (IVF-PQ)** or scalar quantization (int8) to compress index footprint by up to 4x–8x with negligible recall degradation.

Check caption for complete AI Engineer Interview Kit.

2. DATABASES FOR GENAI

Q10. How would you update embeddings when source documents change frequently?

Implement an atomic shadow-ingestion workflow to prevent stale RAG retrievals or downtime during index synchronization.

1. **Priority Webhook Queue:** Listen to source change APIs (Confluence/Drive). Enqueue updates prioritized by document query traffic.

2. **Shadow Ingestion:** Parse, chunk, and embed the updated version into the vector store marked with status `active`.

3. **Atomic Swap & Supersede:** Delete old vector records and update SQL chunks to status `superseded` in a single transaction window.

⚠ CORRECTNESS GUARDRAIL

Always filter vector queries on `status = active` so queries never return duplicate or half-deleted chunks during the update window.

Check caption for complete AI Engineer Interview Kit.

3. RETRIEVAL INFRASTRUCTURE

Q11. What indexing strategies help vector databases perform similarity search efficiently?

Exact k-NN search O(N × D) is too slow for real-time AI. ANN algorithms structure vector space for logarithmic navigation.

ANN Index	Core Data Structure	Best Production Use Case
HNSW	Multi-layer navigable small world graphs	Standard RAG up to 10M–50M vectors (highest recall)
IVF	K-means inverted Voronoi cluster centroids	Fast indexing when memory is constrained
IVF-PQ	Inverted clusters + product quantization	100M+ billion-scale vector collections

CRITICAL HNSW TUNING PARAMETERS

ef_construction: Index quality vs build time • **ef_search**: Query recall vs search latency • **M**: Graph edge density per node.

Check caption for complete AI Engineer Interview Kit.

3. RETRIEVAL INFRASTRUCTURE

Q12. How would you design hybrid search combining vector similarity and keyword search?

Dense embeddings capture semantic meaning; sparse keyword search (BM25) captures exact SKU codes, IDs, and names.

```
RRF_Score = ∑ 1 / (60 + rank_i)
```

HYBRID SEARCH PIPELINE

- **Parallel Retrieval:** Execute dense vector search and sparse BM25 keyword search concurrently across candidate lists.
- **Reciprocal Rank Fusion (RRF):** Merge ranked results without score normalization issues using inverse rank summation.
- **Dynamic Query Weighting:** Boost BM25 weight automatically when exact SKU IDs or proper nouns are detected in the query.

Check caption for complete AI Engineer Interview Kit.

3. RETRIEVAL INFRASTRUCTURE

Q13. What architectural factors influence vector search latency at scale?

End-to-end RAG latency is governed by embedding inference, index memory residency, and payload fetch round-trips.

EMBEDDING API LATENCY

Query embedding generation (50–150ms) frequently dominates total RAG latency over ANN search.

INDEX PLACEMENT

In-memory HNSW responds in <10ms; disk-backed SSD search adds NVMe read latency.

PRE VS POST FILTERING

Metadata post-filtering ruins recall; single-stage HNSW pre-filtering maintains speed.

PAYLOAD HYDRATION

Store chunk text directly or batch-fetch from SQL to prevent N+1 document round-trips.

Check caption for complete AI Engineer Interview Kit.

3. RETRIEVAL INFRASTRUCTURE

Q14. How would you evaluate retrieval quality in a vector database system?

Systematic golden dataset evaluation prevents RAG regressions when changing chunking, models, or index parameters.

Metric	What It Measures	Interpretation
Recall@K	Fraction of relevant docs in top-K	Measures if true evidence is retrieved at all
MRR	Mean Reciprocal Rank of 1st hit	Penalizes systems where correct doc is buried
NDCG@K	Ranked graded relevance score	Rewards having the best document at position 1

ONLINE PRODUCTION TELEMETRY

Complement offline benchmarks by tracking user **query reformulations** and citation click-through rates. Immediate user rephrasing signals poor top-K retrieval.

Check caption for complete AI Engineer Interview Kit.

3. RETRIEVAL INFRASTRUCTURE

Q15. What failure scenarios must be considered when deploying vector search in production?

Vector search failures are frequently silent—degrading RAG accuracy without throwing hard service exceptions.

4 CRITICAL PRODUCTION FAILURE MODES

- **Embedding API Outage:** Implement circuit breakers that fall back to lexical BM25 search when embedding endpoints fail.
- **Silent Index Corruption:** Reconcile vector counts against SQL rows daily and maintain automated snapshot backups.
- **Memory Pressure Eviction:** Monitor OS RSS RAM; page-swapping index segments causes 10x latency spikes.
- **Cold Start Thundering Herd:** Enforce readiness probes so instances fully load HNSW graphs before serving traffic.

Check caption for complete AI Engineer Interview Kit.

CHEAT SHEET

GenAI Memory & Retrieval Decision Matrix

Use this production roadmap to select optimal persistence and indexing architectures for LLM systems.

Problem Requirement	Optimal Technical Solution
Multi-step Working Context	Active Context Window + Step Compression
Long-Running Chat Sessions	SQL History + Recency + Semantic Retrieval
Multi-Tenant Data Privacy	Mandatory Infrastructure Filters (user_id)
SKUs + Semantic RAG Search	Hybrid Search (Dense + BM25) with RRF
Billion-Scale Vector Scale	IVF-PQ / DiskANN + SSD-backed Search

GOLDEN RULES FOR INTERVIEW SUCCESS

- Never pass raw full chat history into LLM context windows.
- Always mention Reciprocal Rank Fusion (RRF) for hybrid search.
- Highlight mandatory tenant ID filters at the DB layer for security.

Check caption for complete AI Engineer Interview Kit.